

# Class 6: R functions

Barry (PID: 911)

## Table of contents

|                                                               |   |
|---------------------------------------------------------------|---|
| Background . . . . .                                          | 1 |
| A first function . . . . .                                    | 1 |
| A <code>generate_dna()</code> function . . . . .              | 3 |
| Write a <code>generate_protein()</code> function . . . . .    | 6 |
| Generate random protein sequences of length 6 to 13 . . . . . | 6 |
| Are Our peptides “unique in nature”? . . . . .                | 7 |
| Connecting our findings to immunology . . . . .               | 8 |

## Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function),
- the *body* (the lines of R code that do the work of the function).

## A first function

Here we will create a function to add some numbers. Let's call it `add()`.

Input arguments can be either “**required**” or “**optional**”. The later have fall-back default values that will be used if the user does not specify them.

**Q1a.** Your first version of the function should add two input numbers together.  
For example, `add(4, 7)` should return 11. [1 pt]

```
add <- function(x, y=0) {  
  x + y  
}
```

Can we use our new function:

```
add(10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

**Q1b:** For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add( c(4,7,10) )`. [1 pt]

```
add <- function(x, y=0) {  
  sum(x,y)  
}
```

```
add(4, 7)
```

```
[1] 11
```

```
add( c(4,7,10) )
```

```
[1] 21
```

**Q1c.** To do on your own... create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)`

```
add <- function(x, y=0, z=0) {  
  sum(x,y,z)  
}
```

```
add(x=1, y=2, z=13)
```

```
[1] 16
```

```
ans <- add(1, 2, 3)
ans
```

```
[1] 6
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the **return()** function call.

```
add <- function(x, y=0, z=0) {
  return(sum(x,y,z))
  cat("Is it break time yet?\n")
}

add(10,100)
```

```
[1] 110
```

## A generate\_dna() function

A useful function here is the “base R” **sample()** function:

```
sample(1:5, size=3)
```

```
[1] 1 3 4
```

```
sample(1:5, size=60, replace = TRUE)
```

```
[1] 5 1 3 1 2 2 1 4 1 2 2 2 4 5 2 4 2 3 1 4 1 3 1 5 3 3 1 2 4 2 2 5 3 2 4 4 3 2
[39] 2 2 5 5 1 1 4 1 5 5 5 1 1 1 2 2 1 1 4 1 3 2
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G” and “T”...

```
sample(x=c("A","C","G","T"), size=10, replace = TRUE)
```

```
[1] "T" "C" "G" "C" "G" "T" "C" "C" "A" "C"
```

**Q2a:** Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”. [1 pt]

```
generate_dna <- function(len=10) {
  sample(x=c("A","C","G","T"), size=len, replace = TRUE)
}
```

```
generate_dna()
```

```
[1] "G" "A" "C" "C" "C" "T" "G" "G" "T" "A"
```

```
generate_dna(len=100)
```

```
[1] "T" "C" "G" "G" "T" "T" "T" "A" "G" "C" "C" "G" "A" "G" "A" "G" "T" "G"
[19] "T" "C" "A" "C" "A" "G" "G" "C" "A" "G" "G" "T" "C" "C" "C" "A" "A" "T"
[37] "A" "C" "C" "C" "C" "T" "A" "T" "G" "C" "T" "C" "A" "C" "C" "C" "A" "A"
[55] "A" "G" "T" "C" "C" "A" "G" "A" "C" "A" "C" "T" "C" "G" "G" "A" "C" "G"
[73] "G" "G" "C" "T" "T" "C" "T" "G" "A" "T" "A" "C" "G" "T" "G" "T" "T" "C"
[91] "C" "C" "A" "A" "A" "G" "C" "A" "T" "C"
```

**Q2b:** Your second version should *optionally* be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”. [1 pt]

```
generate_dna <- function(len=10, single.element=TRUE) {

  ans <- sample(x=c("A","C","G","T"), size=len, replace = TRUE)

  #cat("Hello....")

  if(single.element) {
    # cat("is it me you are looking for...")
    ans <- paste( ans, collapse = "" )
  }
  return(ans)
}
```

Functions that could be useful here are `paste()`, `if()`, `cat()` and `return()`

```
generate_dna()
```

```
[1] "GATTGGCAGA"
```

```
generate_dna(single.element = FALSE)
```

```
[1] "C" "T" "C" "T" "G" "G" "C" "C" "G" "C"
```

**Q2c.** Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
>len9  
CGAAGGCTG
```

```
cat("hello \n there")
```

```
hello  
there
```

```
generate_dna <- function(len=10, single.element=TRUE) {  
  
  ans <- sample(x=c("A","C","G","T"), size=len, replace = TRUE)  
  
  if(single.element) {  
    ans <- paste( ans, collapse = " " )  
  }  
  
  ## Format as FASTA with an ID line  
  cat( paste(">len", len, "\n", sep="") )  
  cat(ans)  
  cat("\n")  
  ##  
  
  return(ans)  
}
```

```
x <- generate_dna(20)
```

```
>len20  
GATTGGCGCTCCATACAACG
```

## Write a `generate_protein()` function

**Q3.** Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return "WQRTAG".

```
generate_protein <- function(len=9) {  
  aa <- c("A", "R", "N", "D", "C", "E", "Q",  
         "G", "H", "I", "L", "K", "M", "F",  
         "P", "S", "T", "W", "Y", "V")  
  
  ans <- sample(x=aa, size=len, replace = TRUE)  
  paste(ans, collapse = "")  
}
```

```
generate_protein(5)
```

```
[1] "CNKKG"
```

## Generate random protein sequences of length 6 to 13

**Q4** Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand.

```
for(l in 6:13) {  
  cat(">", l, "\n", sep="")  
  cat( generate_protein(l), "\n" )  
}
```

```
>6  
FEPFGC  
>7  
ASHSHCR  
>8  
KSPRESCI  
>9  
YDSFFNPDD  
>10
```

```

IWELATMQWG
>11
ENYPQYPMHAQ
>12
EHLWMTSIMPCD
>13
EFKVVPSPGQAAP

```

```
generate_protein(6)
```

```
[1] "ATCNSF"
```

```
generate_protein(7)
```

```
[1] "DRPDVKI"
```

```
generate_protein(8)
```

```
[1] "VALQTEEP"
```

```
generate_protein(9)
```

```
[1] "CAPQHYGDT"
```

## Are Our peptides “unique in nature”?

**Q5:** Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

| length | Ide | Cov | Unique |
|--------|-----|-----|--------|
| 6      | 100 | 100 | N      |
| 7      | 100 | 100 | N      |
| 8      | 100 | 88  | Y      |
| 9      | 100 | 89  | Y      |
| 10     | 90  | 100 | Y      |

| length | Ide | Cov | Unique |
|--------|-----|-----|--------|
| 11     | 90  | 91  | Y      |
| 12     | 77  | 100 | Y      |
| 13     | 100 | 69  | Y      |

### Connecting our findings to immunology

**Q6:** In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]